C 言語基礎編③　テキスト

2021 年 9 月版

明治大学
生田メディア支援事務室

目次

1. 基礎編①②の復習 3
1.1. 変数 3
1.2. 変数の型 4
1.3. 分岐処理（場合分け） 4
1.3.1. if 文 5
1.3.2. switch 文 6
1.4. 反復処理 7
1.4.1. for 文 7
1.4.2. while 文 7
2. 配列 8
2.1. 1 次元配列 8
2.2. 多次元配列 10
2.3. 演習問題 11
3. 関数 13
3.1. 関数の定義と呼び出し 13
3.2. 関数の宣言 18
3.3. 変数の種類 19
3.4. 演習問題 21
4. ファイル分割 24
4.1. ソースファイル 24
4.2. ヘッダファイル 24
4.3. ヘッダファイルインクルード 24
4.4. ファイル結合 25
5. 参考文献 28
6. 演習問題解答例 29
6.1. 2 章演習問題 29
6.2. 3 章演習問題 31

1. 基礎編①②の復習

C 言語とは、ソフトウェアを作る際に用いるプログラミング言語の 1 種です。他にも様々なプログラミング言語がありますが（Java、Python、Ruby など）、C 言語には以下の特徴があります。

1. プラットフォーム※に依存しないため、プログラムの移植性が高い
2. 機械語に翻訳した実行ファイルを作成（コンパイル）したのち、プログラムを実行するため、実行速度が速い

また、他のプログラミング言語は C 言語を意識して作られたものが多く、これを覚えるだけで他の言語も習得しやすくなるメリットがあります。

本章では、本講習会で学ぶ「配列」や「関数」に関する前提知識となる C 言語基礎編①②で学習した内容を、簡単に復習しておきます。

1.1. 変数

変数とは、メモリに割り振った名前のことです。メモリは縦 1 列に膨大に並べられたロッカーのような構造をしており、その 1 つ 1 つに番号（アドレス）が割り振られています。ソフトウェアはこのメモリの中にあるデータを出し入れして、演算処理を行っています。もし、メモリのアドレスを指定してデータの保存や使用を行おうとしても、人間では手に余ります。そこで「変数の宣言」を行います。これにより、変数の名前と適当なメモリのアドレスをコンピュータが関連付け、変数の名前を呼び出すだけで指定のメモリを扱えるようになります（図 1）。

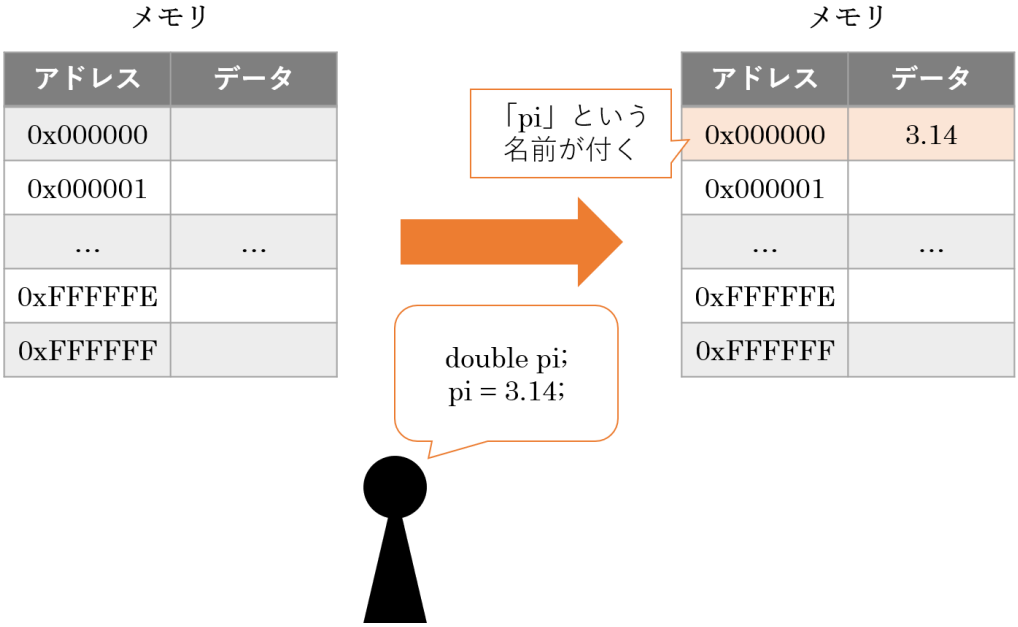


図1.　メモリの構造と変数宣言

※ ここでは主に OS のことを指します

1.2.　変数の型

保存するデータによって、型を選ぶ必要があります。char は主に文字を保存しますが、数値も保存できます。int は、整数値を扱うための型です。4 バイトであるため、1 バイトの char と比べてより大きい範囲の数値を保存できます。double 型は float 型と同じ実数を扱うことができる型ですが、float 型に比べてより大きい範囲の値を保存できます。表 1 に変数の型と保存できる数値の範囲を示します。

表 1.　変数の型と保存できる値の範囲

種類	型名	サイズ	値の範囲
文字型	char	1 バイト	-128 ～ 127
整数型	short int	2 バイト	-32768 ～ 32767
	int	4 バイト	-2147483648 ～ 2147483647
浮動小数点型	float	4 バイト	3.4E-38 ～ 3.4E+38
（倍精度）浮動小数点型	double	8 バイト	1.7E-308 ～ 1.7E+308

また、変数の型によって printf 関数や scanf 関数を用いる際の変換指定子の記述方法が変化します。表 2 に変数の型と変換指定子の対応を示します。

表 2.　変数の型と変換指定子

種類	型名	printf 関数	scanf 関数
文字型	char	%s	%s
整数型	short int	%hd	%d
	int	%d	%d
浮動小数点型	float	%f	%f
	double	%f	%lf

1.3.　分岐処理（場合分け）

プログラムの最中で、条件によって実行する内容を変えたいときがあります。そういうときには「分岐処理」を用います。C 言語の分岐処理には if 文と switch 文があります。switch 文で書けることは if 文でも行えますが、逆は行えないことが多いので、主に if 文が用いられます。それでも switch 文の方が可読性に優れることもあるので、適宜使い分けることが大切です。

1.3.1.　　if 文

① 単純 if 型（もし～ならば、○○○を行う）

条件式を確認し、その内容が正しければ処理を実行します。処理が複数ある場合には、全体を{}で囲う必要があります。

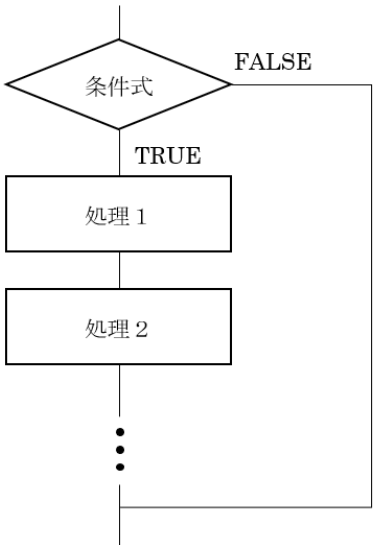


図2. if 文のフローチャート

構文

```
if（条件式）{
                処理１；
                処理２；
                  …
}
```

例）

```
int a = 6;
if（a > 5 && a < 10）{
                printf("aは5より大きい");
                printf("aは10より小さい");
}
```

実行結果

```
aは5より大きい
aは10より小さい
```

② if－else 型（もし～ならば○○○を行い、そうでないならば×××を行う）

if 文の後に else if と続けることで、複数の条件に分岐させることができます。最後は else のみで終わらせなければなりません。

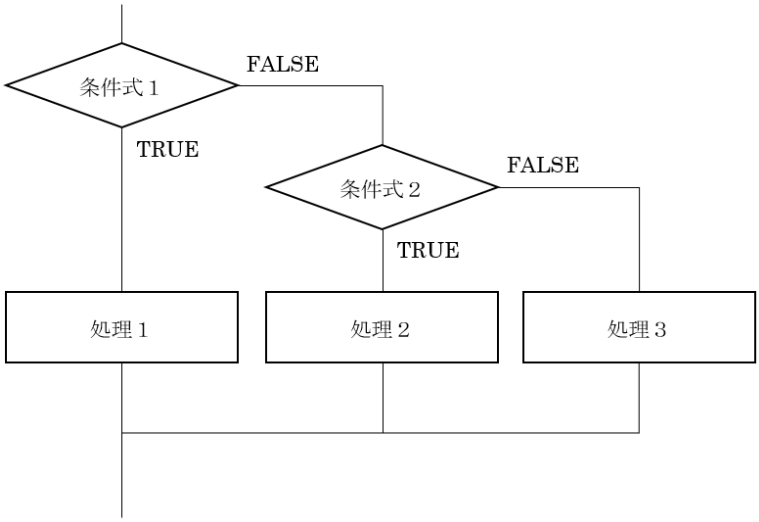


図3. if-else 文のフローチャート

構文

```
if（条件式１）{
                処理１；
} else if（条件式２）{
                処理２；
} else {
                処理３；
}
```

例）

```
int a = 6;
if（a > 10) {
                printf("aは10より大きい");
} else if (a > 5) {
                printf("aは5より大きくて10以下);
} else {
                printf("aは5より小さい");
}
```

実行結果

```
aは5より大きくて10以下
```

1.3.2. switch 文

if 文と同様に、条件によって処理を変えるときに使用します。式の中身がラベルと一致している場合の処理を実行します。ラベルは、整数値か「A」のような 1 文字のみ使用可能です。

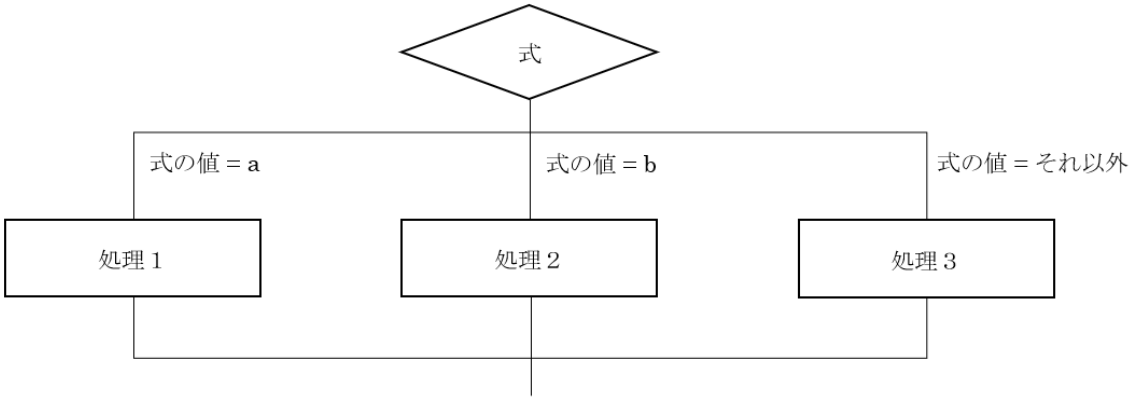


図4. switch 文のフローチャート

構文

```
switch（式）{
        case a ：
                処理 1 ；
                break ;
        case b ：
                処理 2 ；
                break ;
        default ：
                処理 3 ；
                break ;
}
```

例）

```
int a = 2;
switch (a)
 {
        case 1:
                printf("aは1です¥n");
                break;
        case 2:
                printf("aは2です¥n");
                break;
        default:
                printf("aは1,2以外です¥n");
                break;
}
```

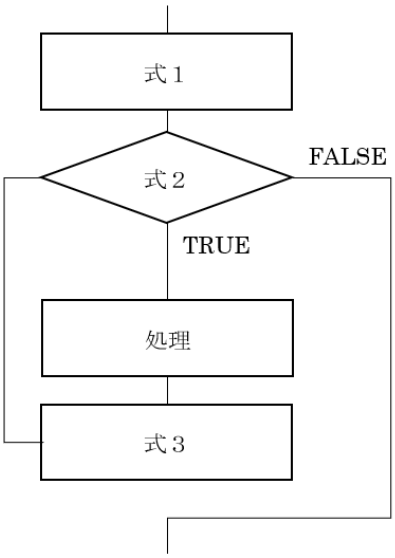
実行結果

```
aは2です
```

1.4. 反復処理

同じ処理を何度も記述するのが面倒なときには、「反復処理」を用います。for 文と while 文があり、どちらもよく使われます。for 文でできることは while 文でもできますし、逆も同様です。しかし、for 文は、主に繰り返す回数が決まっているときに使用します。

1.4.1. for 文



構文

```
for（式１；式２；式３）{
                処理；
}
```

例）

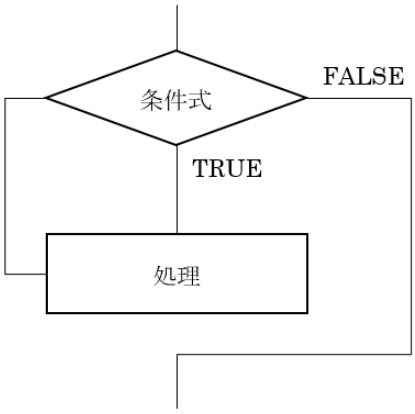
```
for (i = 0; i < 2; i++){
                printf("iは%dです¥n", i);
}
```

実行結果

```
iは0です
iは1です
```

図5. for 文のフローチャート

1.4.2. while 文



構文

```
while（条件式）{
                処理；
}
```

例）

```
int i = 2;
while (i > 0) {
                printf("iは%dです¥n", i);
                i--;
}
```

実行結果

```
iは2です
iは1です
```

図6. while 文のフローチャート

2. 配列

配列（array）は、変数のリスト（並び）のことをいいます。プログラムでは、単一の変数を 1 つ 1 つ宣言して使用するよりも、複数の変数の並び（＝配列）を用いた方が使いやすいことが多くあり、その代表に文字列があります。文字列は、1 つ 1 つの文字の並びからできています。これを、char 型をいくつも宣言していてはとても使いやすいとはいえません。しかし、文字の一連の並びをまとめて宣言できれば、とても使いやすくなります。C 言語では、これを配列を用いて実現することができます。

2.1. 1 次元配列

配列とは変数のリストであり、1 度に複数の変数を宣言できます（ただし、型が異なる変数はそれぞれ宣言する必要があります）。以下に 1 次元配列の宣言及びアクセス方法を示します。

＜宣言＞

```
型名　変数名［要素数］;
```

例）

```
int month[12];
//int 型の month という変数が 12 個生成される
```

＜アクセス＞

```
変数名［アクセスしたい要素番号］= 値 ;
```

例）

```
month[0] = 31
//変数名とアクセスしたい要素番号を指定する
//month という変数の 1 番目に 31 が代入される
```

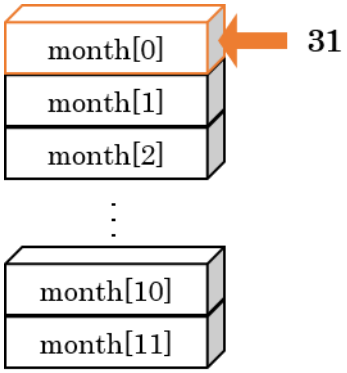


図7. 1 次元配列の概念図

また、配列に初期値を代入する方法として、以下の 3 通りの方法があります。

```
int month[12] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
//各要素の値をカンマ( , )で区切り、最初と最後を{}で囲む

int month[12] = { 31, 28, 31, 30};
//要素数と初期値の数が異なる場合、値が指定されなかった要素には自動的に 0 が代入される
//この場合は、month[4]以降はすべて 0 となる

int month[ ] = { 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
//要素数を指定しない場合、初期値の数が自動的に要素数に指定される
//この場合は、int month[12]と宣言したことと同じになる
```

ここで、注意すべきポイントとして、以下の 3 点が挙げられます。

【1】: 要素数が N 個の配列の要素番号は、0 から（N－1）まで。

例）

```
int month[12];
month[0] = 31;
month[1] = 28;
        ・
        ・
month[11] = 31;
//要素番号は（要素数－1）まで
```

【2】: 定義されている要素数以上の要素番号にアクセスしても、コンパイルエラーにはならないが、実行時に不具合が発生する可能性がある。

例）

```
int month[12];
int i;
for( i = 0; i < 100; i++){
        month[i] = i + 1;
}
//実行はできるが、問題が起きる可能性が大きい
```

【3】: まとめてのコピーは不可。

例）

```
int num1[12], num2[12];
num1 = num2;
//配列そのものを「=」で結ぶとエラー発生
```

2.2.　多次元配列

配列には 1 次元配列だけでなく、多次元の配列も存在します。以下に 2 次元配列を例として、多次元配列の宣言及びアクセス方法を示します。

＜宣言＞

```
型名　変数名［行数］［列数］;
```

例）

```
int matrix[4][3];
//matrix という 4 行 3 列の 2 次元配列を宣言
```

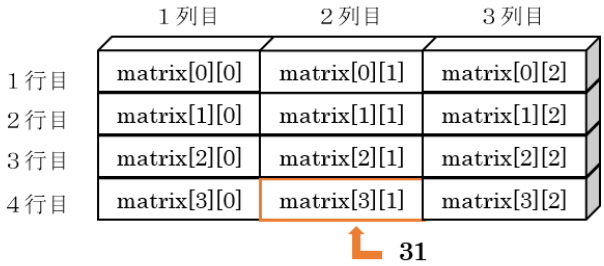


図8.　2 次元配列の概念図

＜アクセス＞

```
変数名［アクセス先の要素番号］［アクセス先の要素番号］= 値 ;
```

例）

```
matrix[3][1] = 31;
//matrix の 4 行 2 列目に 31 を代入
```

また、初期値の代入は以下のように行います。

```
int matrix[4][3] = { { 11, 12, 13},
                     { 21, 22, 23},
                     { 31, 32, 33},
                     { 41, 42, 43} }
//行ごとに要素の値を{}で囲んで指定し、それらをさらにカンマ( , )で区切る
```

2 次元配列の場合も、1 次元配列と同様に、初期値を省略した場合には、残りの要素にはすべて 0 が代入されます。また、要素数も省略することができますが、2 次元配列の場合、省略できるのは、1 つ目の要素数（行数）のみになります。2 つ目の要素数（列数）を省略すると、エラーが発生しますので注意してください。

```
int matrix[ ][3] = { { 11, 12, 13},
                     …            }  //このような省略は可能

int matrix[4][ ] = { { 11, 12, 13},
                     …            }  //これはエラー発生
```

また、3 次元以上の場合も、

```
int matrix_3d[4][3][2];
```

のように、[] の数を増やすことで定義することができます。

2.3.　演習問題

【1】

以下の条件を満たすプログラム「count_sentence.c」を作成するために、網掛け部分を埋めてみましょう。

・文字列をキーボードから入力して、配列に格納する

・その文字列を画面に表示する

・その文字列の文字数をカウントし、表示する

＜count_sentence.c＞

```
#include <stdio.h>

int main(void)
{
        str[100];
        int count;
        //入力した文字列を配列に格納
        printf( “文字を入力してください : ”);
        scanf( “”, );
        //入力した文字列を表示
        printf( “文字列は「」です¥n”, );

        count = ;
        while( str[] != ‘¥0’){   //注釈 1
                ;
        }
        printf( “文字数はです¥n”, );

        return 0;
}
```

※注釈 1 : 条件文に使用されている「¥0」は、「null 文字」といって、文字列の終端を示すための値です。配列に文字列が格納されたとき、コンパイラによって自動的に、文字列の最後にこの null 文字が格納されます。したがってこの部分では、「文字列の先頭から各要素をたどっていき、文字列の終端（null 文字）が現れるまで実行を繰り返す」という条件を設定することで、問題文に沿うような処理が実行できます。

null 文字についてさらに詳しく知りたい方は、以下のリンクを参考にしてください。

（https://edu.clipper.co.jp/pg-2-16.html）

【2】

以下の条件を満たすプログラム「kuku.c」を作成するために、網掛け部分を埋めてみましょう。

・2 次元配列 kuku に、九九の計算結果を格納する

・kuku の値を出力し、九九の表を画面に表示する

なお、実行結果のイメージは下の図を参考にしてください。

＜実行結果イメージ＞

```
 1  2  3  4  5  6  7  8  9
 2  4  6  8 10 12 14 16 18
 3  6  9 12 15 18 21 24 27
 4  8 12 16 20 24 28 32 36
 5 10 15 20 25 30 35 40 45
 6 12 18 24 30 36 42 48 54
 7 14 21 28 35 42 49 56 63
 8 16 24 32 40 48 56 64 72
 9 18 27 36 45 54 63 72 81
```

＜kuku.c＞

```
#include <stdio.h>

int main(void)
{
        kuku      ;   //2 次元配列を定義
        int i, j;

        for( i = 0; i <  ; i++){
                for( j = 0; j <  ; j++){
                        kuku    =          ;   //2 次元配列に答えを格納
                        printf( "    ",          );   //九九の計算結果を画面に表示
                }
                printf( "  ");
        }

        return 0;
}
```

3. 関数

3.1. 関数の定義と呼び出し

関数とは、一言でいうと「関連する複数の処理を１つにまとめたプログラム」です。
数百行や数千行というプログラムを書く場合、どの部分でどのような処理が行われているのか把握できていないと、プログラムが不具合を起こしてしまう可能性があります。そこで、関数を用いて、関連した処理（累乗を計算する処理や総和を計算する処理など）を 1 つにまとめることで、プログラム全体がコンパクトになり、ミスの修正もしやすくなります。

関数を用いてプログラムをまとめる利点は、次の 3 点です。

・プログラムがコンパクトになり、処理が把握しやすくなる
・同じような処理を複数回記述する手間を省ける
・各機能を関数としてまとめておくことで、プログラムの修正が容易になる

関数の中身を自身で作成することを「関数を定義する」、定義した関数を実際に用いることを「関数を呼び出す」といいます。

以下に、関数の定義及び呼び出し方法を示します。

＜定義＞

```
戻り値の型　関数名(型名　変数名，　型名　配列名［］，　・・・){
        ・・・
        関数内での処理；
        ・・・
        return 文；　//戻り値
}
```

＜呼び出し＞

```
int main(void)　//メイン文
{
        ・・・
        関数名（引数名，　配列名，　・・・）；　//関数呼び出し　引数の型は指定しなくてよい
        ・・・
}
```

関数名の後の（）内に記述されている変数や配列のことを「引数」と呼びます。引数とは、関数の中身に渡す何らかの情報（値）のことで、引数を受け取った関数は、その値を用いて処理を実行します。

なお、引数の数に制限はありません。引数をいくつ用いても構いませんし、逆に引数のない関数を定義することも可能です。その場合には、引数に「void」を指定します。

関数の本体で定義されている引数を「仮引数」、関数の呼び出し元から渡される引数を「実引数」と呼びます。仮引数の指定には、引数の型を指定する必要がありますが、実引数には型を記述する必要はありません。

また、仮引数に配列を指定するとき、要素数を省略することができます（要素数を指定することもできますが、汎用性の低い関数になってしまいます）。ただし、仮引数が配列である、ということは指定しないといけないため、「配列名［］」と記述します。逆に、実引数には「配列名」のみを記述します。実引数に「配列名［］」と指定すると、エラーが発生するので気を付けましょう（例 2）。

さらに、関数での処理の実行後に、その関数が結果として出力する情報（値）のことを、「戻り値」といいます。引数と異なり、戻り値は 1 つのみ指定することができます。関数は、処理が一通り完了すると、最後に return 文の値を呼び出し元に返します。関数の定義では、この戻り値の型も指定する必要がありますが、戻り値のない関数を定義することもでき、その場合は戻り値の型を「void」として、return 文は省略します（例 3）。

以上の説明をまとめたものを表に示します。表にまとめられた要素により関数は構成されており、定義の際にこれらの要素に漏れがあると、プログラムが思うように動作しないこともあります。

表 3.　関数の要素

要素名	説明
関数名	その関数の名前
引数	関数内の処理で用いるための情報（値）
処理	関数内で行いたい処理（実際の動作）
戻り値	関数が呼び出されたとき、最終的に呼び出し元に返す情報（値）
戻り値の型	戻り値がどのような型の情報であるか

例 1）2 つの引数の和の値を計算する関数 wa（引数の型は、どちらも整数とする）。

＜sample1.c＞

```
#include <stdio.h>

int wa( int x, int y){   //戻り値の型、及び仮引数を指定
        return (x + y);   //x+y の値を結果として返す     () は省略可能
}

int main(void)
{
        int x_input = 1;
        int y_input = 2;

        int sum = wa( x_input, y_input);

        printf( “sum = %d¥n”, sum);

        return 0;
}

//実行結果：sum = 3
```

関数 wa では

1. x, y という 2 つの仮引数を取る
2. 戻り値として x と y の和を返す

という処理を行っています。

また、main 関数では

1. x_input = 1、y_input = 2 をそれぞれ代入する
2. 2 つの変数 x_input, y_input を実引数として関数 wa に渡す
3. 変数 sum に関数 wa の戻り値を代入する
4. sum の値を画面に出力する

という処理を行っています。

例 2）任意の人数のテストの結果を受け取りその平均点を出力する関数 avg

＜sample2.c＞

```
#include <stdio.h>

float avg( int test[ ], int num){
        int i;
        float sum = 0;

        for( i = 0; i < num; i++){
                sum += test[i];
        }
        return (sum/num);
}

int main(void)
{
        int test[5] = { 56, 78, 98, 31, 69};
        printf( “5 人の平均点は%.2f 点です¥n”, avg(test, 5));

        return 0;
}

//実行結果：5 人の平均点は 66.40 点です
```

関数 avg では

1. test という配列と num という整数の 2 つの引数を取る
2. 配列 test には num 人分のテスト結果が格納されており、それらの値を変数 sum に足し合わせる
3. 最後に sum の値を num で割った値を返す

という処理を行っています。

main 関数では

1. 配列 test に 5 人分のテストの結果を格納する
2. 実引数に配列 test と、5 という数値を渡して結果を画面に出力する

という処理を行っています。

例 3）2 つの引数の大小関係を比較し大きいほうの値を画面に出力する関数 compare
＜sample3.c＞

```
#include <stdio.h>

void compare( int x, int y){
	if( x > y){
		printf( “%d の方が大きい¥n”, x);
	} else if( x < y){
		printf( “%d の方が大きい¥n”, y);
	} else {
		printf( “2 つの値は等しい¥n”);
	}
}	//void 型の場合、return 文は省略できる

int main (void){
	int x_input = 1;
	int y_input = 2;
	compare( x_input, y_input);

	return 0;
}

//実行結果：2 の方が大きい
```

関数 compare では

1. x, y という 2 つの仮引数を取る
2. 2 つの数の大小関係を比較し、
   i） x > y ならば、「x の方が大きい」と画面に出力する
   ii） x < y ならば、「y の方が大きい」と画面に出力する
   iii）x = y ならば、「2 つの値は等しい」と画面に出力する

という処理を行っています。このとき、関数 compare は“画面に出力する”という動作を定義していますが、この関数そのものが何か具体的な値を返すようなことはしません。このように関数自体が戻り値を持たない関数を、「void 型関数」といいます。

main 関数では

1. x_input = 1, y_input = 2 をそれぞれ代入する
2. 関数 compare を呼び出して、2 つの変数 x_input, y_input を実引数として渡す

という処理を行っています。

3.2.　関数の宣言

C 言語には、「呼び出す関数を使用する前に定義しておく必要がある」というルールがあります。しかし、プログラム全体が複雑になり関数の数も増えてくると、関数の呼び出し順序を整理するのに手間がかかることもあります。このような事態に対応するため、C 言語にはプロトタイプ宣言という構文が用意されています。これは、先に関数の名前・戻り値の型・引数を宣言しておき、処理の部分は後ほど記述するという方法です。この構文を用いることで、関数の本体をコード内のどこに記述していても、関数を呼び出すことが可能になります。表 4 に、関数の宣言と定義の違いをまとめます。

表 4.　関数の宣言と定義

関数の「宣言」	関数の名前・戻り値の型・引数のみを指定する。「中身は後で決めるけれど、ひとまずこういう関数がありますよ」とコンピュータに“宣言”するときに用いる。
関数の「定義」	関数の名前・戻り値の型・引数とともに、中身の処理まで記述する。その関数でどのような処理がしたいのかを“定義”するときに用いる。

＜プロトタイプ宣言＞

```
戻り値の型　関数名（引数リスト）;
```

＜sample4.c＞

```
#include <stdio.h>

int wa( int x, int y);　//関数 wa のプロトタイプ宣言

int main(void)
{
        int input1 = 1, input2 = 2;
        //関数 wa の呼び出し
        printf( “%d + %d = %d¥n”, input1, input2, wa(input1, input2));
        return 0;
}
//プロトタイプ宣言によって、呼び出しより後に定義をしてもエラーが起きない
int wa( int x, int y){
        return x+y;
}

//実行結果：1 + 2 = 3
```

3.3.　変数の種類

ここで、sample2.c（15 ページ）を思い出してみましょう。avg という関数を作成して、num 人のテストの平均点を計算していました。ここで、関数 avg 内において「sum」という変数には、テストの合計点が格納されています。では、この「sum」は、どの範囲まで有効なのでしょうか。例えば、5 人のテストの合計点を知りたいとき、main 関数の中で変数 sum を参照することはできるのでしょうか。答えは No です。関数内で宣言された変数は、**その関数内でしか**使用することができません。sum だけでなく、「i」も関数 avg の外で用いることはできません。

＜sample5.c　（sample2.c を一部変更）＞

```
#include <stdio.h>

float avg( int test[ ], int num){
        int i;
        float sum = 0;

        for( i = 0; i < num; i++){
                sum += test[i];
        }
        return (sum/num);
}

int main(void)
{
        int test[5] = { 56, 78, 98, 31, 69};
        printf( “5 人の平均点は%.2f 点です¥n”, avg(test, 5));
        printf( “5 人の合計点は%.2f 点です¥n”, sum);
        //main 関数内で宣言していないためエラー発生

        return 0;
}
```

このように、関数内で宣言された変数を「ローカル変数」といいます。一方で、変数を関数の外で宣言することもできます。変数外で宣言された変数を「グローバル変数」といいます。本節では、この 2 つの変数の違いについて説明していきます。

ローカル変数とグローバル変数の違いを説明するため、簡略的なソースコードを sample6.c に示します。

＜sample6.c＞

```
#include <stdio.h>

int global_a;   //グローバル変数

void func( void){
        int local_b;   //ローカル変数
        ・・・
}

int main(void)
{
        int local_c = 1;   //ローカル変数
        ・・・
        func( );

        return 0;
}
```

global_a の有効範囲（スコープ）

local_b の有効範囲（スコープ）

local_c の有効範囲（スコープ）

sample6.c では、func という関数と main 関数の中で、それぞれ local_b, local_c というローカル変数を宣言しています。また、2 つの関数の範囲外の場所で、global_a というグローバル変数を宣言しています。このとき、それぞれの変数の有効範囲は四角で囲った範囲までとなっており、global_a はプログラム全体で有効なのに対して、local_b, local_c はそれぞれが定義されている関数の中でしか用いることができません。このように、変数にはそれぞれその変数を用いることのできる有効範囲が決まっており、この範囲のことを「スコープ」と呼びます。

sample5.c においても、例えば変数 sum をグローバル変数として宣言することで、main 関数の中でも sum の値を参照することができるようになります。ただし裏を返せば、思わぬところで sum の値が更新されてしまうという危険性も潜んでいます。グローバル変数の扱いには十分な注意が必要です。

大規模なプログラムになるほど、グローバル変数を使いこなすのは難しくなります。しかし、こういった変数も存在する、という知識を持っておくだけで、他人が書いたソースコードを読み解いたり、不具合の原因を探したりするときに役に立ちます。ローカル変数とグローバル変数のそれぞれの特徴について、しっかりと理解しておきましょう。

3.4.　演習問題

【1】

以下の条件を満たすプログラム「circle.c」を作成するために、網掛け部分を埋めてみましょう。

・半径の値を引数に取り、面積を返す関数 area_of_circle を作成する

・main 関数で area_of_circle を呼び出し、キーボードで入力した半径を持つ円の面積を表示する

＜circle.c＞

```
#include <stdio.h>
#include <math.h>   //注釈 1
//関数定義
                                    {
        return                  * M_PI;   //注釈 2
}

int main(void)
{
        radius;   //半径を格納する変数を宣言
        //半径の値をキーボードから入力
        printf( “radius = ”);
        scanf(                  );
        //関数を呼び出して結果を画面に表示
        printf( “面積は   です¥n”,                    );

        return 0;
}
```

※注釈 1：「math.h」とは、基本的な数学演算を行うためのヘッダファイルです。なお、ヘッダファイルに関しては、4 章で詳しく説明します。ここでは、「プログラムの実行に必要なツールが別ファイルに記述されたもの」と考えてください。

※注釈 2：「M_PI」とは、math.h の中で円周率の値として定義されている変数です。

【2】

以下の条件を満たすプログラム「sum_of_squares.c」を作成するために、網掛け部分を埋めてみましょう。

・引数の 2 乗の値を求める関数を作成する（引数は整数とする）

・3 つの引数の和を求める関数を作成する

・上で作成した 2 つの関数を使用して、キーボードから入力した 3 つの値の 2 乗和を計算し、結果を画面に表示する（例えば入力が x, y, z のとき、x^2+y^2+z^2 を計算する）

＜sum_of_squares.c＞

```
#include <stdio.h>

    jijo (          ){   //引数の 2 乗を求める関数
            ;
}

    wa_3 (                                  ){   //3 つの数の和を求める関数
            ;
}

int main(void)
{
            input[3] ;   //キーボードから入力した値を格納する変数を配列で定義

            int i;
            for( i = 0; i < 3; i++){   //キーボードから 3 つの値を入力
                    printf( “input%d : ”, i+1);
                    scanf(                 );
            }
            //関数を呼び出して計算結果を answer に格納
            answer =                                        ;
            //結果を画面に表示
            printf( “    ^2 +    ^2 +    ^2 =    ¥n”,          ,          ,          ,          );

            return 0;
}
```

【3】

以下の条件を満たすプログラム「sum_of_list.c」を作成するために、網掛け部分を埋めてみましょう。

・【2】で作成した、関数 jijo 及び関数 wa_3 の機能を併せ持つ新たな関数 jijo_wa を作成する

・jijo_wa は、整数型の配列と、その配列の要素数を引数に取る

・関数 jijo_wa を使用して、キーボードから入力した 3 つの値の 2 乗和を計算し、結果を画面に表示する

＜sum_of_squares_2.c＞

```
#include <stdio.h>

    jijo_wa (                    ){   //配列の全要素の 2 乗和を求める関数
                                                                    
                                                                    
                                                                    
}

int main(void)
{
        input[3] ;   //キーボードから入力した値を格納する変数を配列で定義

        int i;
        for( i = 0; i < 3; i++){   //キーボードから 3 つの値を入力
                printf( “input%d : ”, i+1);
                scanf(                 );
        }
        //関数を呼び出して計算結果を answer に格納
        answer =                 ;
        //結果を画面に表示
        printf( “    ^2 +    ^2 +    ^2 =    ¥n”,                                    );

        return 0;
}
```

4. ファイル分割

本講習会で学んだ関数や変数の性質を用いることで、C 言語をより効率良く、柔軟に扱っていくことができます。これらの機能は非常に便利ですが、プログラムの規模が大きくなったときのことを考えてみましょう。例えば、自身でいくつもの関数を定義するとき、それらを定義するソースコードが、すべて main 関数と同じファイルに記述されていたらどうでしょうか。コード全体をコンパクトに、見やすくするために関数を定義しているのに、その関数自体の中身が膨大で、main 関数が埋もれてしまうようなことがあるかもしれません。あるいは、あるプログラムで作成した関数を、別のプログラムで使い回したい場面が出てくるかもしれません。

そのような事態に対応するため、C 言語には「処理を複数のファイルに分割して記述し、実行の際に 1 つのファイルにまとめて実行する」という便利な機能が備わっています。この機能を用いることで、関数の定義のみを記述するファイル、main 関数の処理を記述するファイル、のようにそれぞれの役割ごとにファイルを分割することができます。本章ではこのようなファイルの分割や、それらを連結させて実行する方法について学習していきます。

ファイルを分割して記述する際、記述する内容によってファイルの種類を変える必要があります。まず、C 言語で用いられる「ソースファイル」と「ヘッダファイル」という 2 種類のファイルについて、違いをそれぞれ説明します。

4.1. ソースファイル

ソースファイルとは、これまでの講習会で皆さんがたくさん作成してきたような、ファイル名の末尾に「.c」がつくタイプのファイルです。これまでこのファイルには、「実行したい処理の内容（main 関数）」を必ず記述していましたが、「関数の定義」のみを記述することも可能です。他にも、「グローバル変数の定義」、「ヘッダファイルインクルード[1]」、「マクロ定義[2]」などがソースファイルに記述されます。

4.2. ヘッダファイル

ヘッダファイルとは、ソースと同様の形式をしていて、「ソースファイルである機能を用いたいときに必要な情報を記述する」ファイルです。ファイル名の末尾には「.h」を指定します。この種類のファイルには、「関数のプロトタイプ宣言」、「グローバル変数の宣言」、「ヘッダファイルインクルード」、「マクロ定義」、「構造体型の宣言・定義[2]」などを記述します。

4.3. ヘッダファイルインクルード

ヘッダファイルに書き込んだ内容を別のソースファイルで読み込みたいとき、C 言語の「include」という機能を用います。include を用いてソースファイルにヘッダファイルを読み込むことを、「ヘッダフ

[1] 4.3 節で説明
[2] C 言語基礎編④で説明

ァイルインクルード」と呼びます。プログラムの先頭に必ず記述している、「#include <stdio.h>」が代表的な例です。「stdio」は「standard input/output」の略で、stdio.h には、標準入出力を制御するために必要な基本的な関数、記号定数、マクロなどの定義が記述されています。このヘッダファイルをインクルードすることで、printf や scanf などの入出力操作を行う関数を用いることができるようになります。stdio.h の中身には、1,000 行弱のソースコードが記述されていますが、これらを毎回 main 関数と同じファイルに記述するのはあまりにも非効率的です。そのため、必要な情報が記述されたファイルをインクルードするという機能が備わっています。

printf や scanf のような、C 言語に標準で備わっている関数を「標準ライブラリ関数」と呼び、これらの定義などが記述されたヘッダファイルを「標準ヘッダファイル」と呼びます。標準ヘッダファイルをインクルードするときは、ファイル名を「＜＞」で囲みます。一方で、自身で作成したヘッダファイルをインクルードするときは、ファイル名を「“”」で囲みます。インクルード方法の違いに気を付けましょう。

```
#include <stdio.h>   //標準ヘッダファイルのインクルード
#include “my_header.h”   //自作ヘッダファイルのインクルード
```

4.4. ファイル結合

ここまで、複数ファイルに処理を分割して記述する際に、必要となる基礎的な知識について説明してきました。ここからはいよいよ、複数ファイルを連結させて実行する手順について説明します。ヘッダファイルに関しては、インクルードを用いることで別ファイルに記述した内容を読み込ませることができました。しかしながら、ソースファイルをインクルードすることはできません。したがって、ソースファイルを複数用意する場合には、ターミナル上で実行ファイルを作成するときに、それぞれのソースファイルを連結させて実行ファイルを作成します。

実際のターミナル上でのコマンドを説明します。複数ファイルを連結させる際、まずそれぞれのソースファイルを機械語に翻訳した、「オブジェクトファイル」というファイルを作成する必要があります。オブジェクトファイルの作成は以下のコマンドで行えます。

```
gcc -c ファイル名 1.c ファイル名 2.c ・・・
```

コンパイル時に用いていた gcc コマンドに、「-c」というオプションを指定することで、オブジェクトファイルを作成することができます。オブジェクトファイルは末尾が「.o」となっており、例えば「test.c」というソースファイルからオブジェクトファイルを作成した場合、ファイル名は「test.o」となります。

次に、これらのオブジェクトファイルを 1 つにまとめて実行ファイルを作成します。 これまで通り「-o」というオプションを指定することで、実行ファイルを作成することができます。

```
gcc ファイル名 1.o ファイル名 2.o ・・・ -o 実行ファイル名
```

例として、sample5.c（本テキスト 18 ページ）をもとに、「main.c」と「func_define.c」という 2 つのソースファイル及び、「func_declare.h」というヘッダファイルをそれぞれ作成し、これらを連結させて実行します。

【複数人のテスト結果の平均点と合計点を表示するプログラム】

＜func_declare.h＞

```
#include <stdio.h>

float sum;   //グローバル変数 sum の宣言
float avg( int test[], int num);   //関数 avg のプロトタイプ宣言
```

＜func_define.c＞

```
#include “func_declare.h”   //ヘッダファイルインクルード

float avg( int test[ ], int num){   //関数 avg の定義
        sum = 0;
        int i;
        for( i = 0; i < num; i++){
                sum += test[i];   //グローバル変数 sum の定義
        }
        return (sum/num);
}
```

＜main.c＞

```
#include “func_declare.h”

int main(void)
{
        int test[5] = { 56, 78, 98, 31, 69};
        printf( “5 人の平均点は%.2f 点です¥n”, avg(test, 5));   //関数 avg の呼び出し
        printf( “5 人の合計点は%.2f 点です¥n”, sum);   //グローバル変数 sum の参照

        return 0;
}
```

【実行画面】

```
[                    ]% gcc -c main.c func_define.c
[                    ]% gcc main.o func_define.o -o out
[                    ]% ./out
5 人の平均点は66.40点です
5 人の合計点は332.00点です
```

func_declare.h では、テスト結果の合計点を格納する変数 sum、そして、平均点を計算する関数 avg の宣言をそれぞれ行っています。func_define.c では、func_declare.h をインクルードして関数 avg の処理を定義しています。func_declare.h で、すでに stdio.h のインクルードを行っているため、func_define.c 内での stdio.h のインクルードは行っていません。main.c でも同様に func_declare.h のインクルードをした後、関数 avg の呼び出しと変数 sum の参照を行うことで、平均点と合計点を表示させています。

プログラムの実行時には、main.c と func_define.c から、それぞれのオブジェクトファイルを作成し、それらを連結させて「out」という実行ファイルを作成し、実行しています。

図 9 にプログラム実行までのイメージ図を示しますので、参考にしてください。

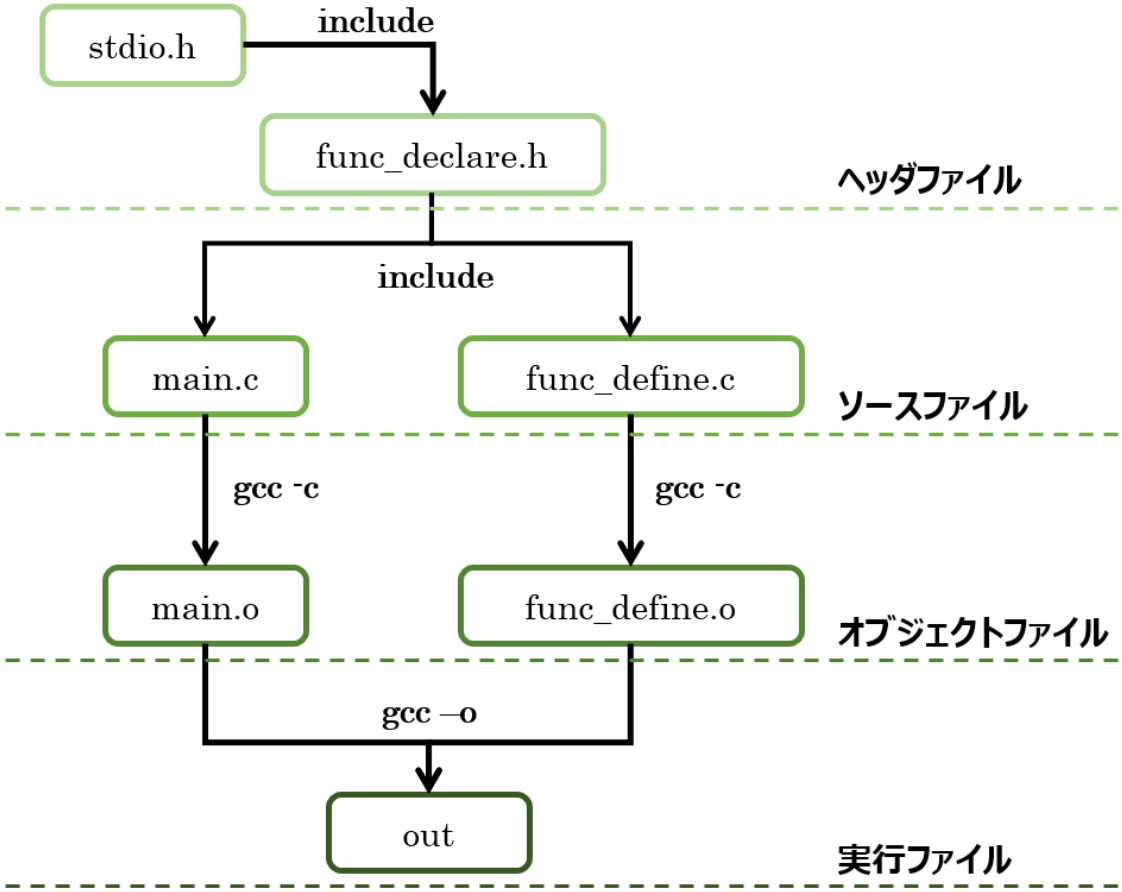


図9. ファイル連結のイメージ図

5. 参考文献

[1]. 『やさしい C』, 高橋麻奈, ソフトバンクパブリッシング株式会社, 2003 年
[2]. 『明解 C 言語　入門編』, 柴田望洋, ソフトバンククリエイティブ株式会社, 2008 年
[3]. 『文字列の終端 null 文字（'¥0'）に関すること』, (https://edu.clipper.co.jp/pg-2-16.html)
[4]. 『C 言語 ヘッダファイルの書き方』, (https://monozukuri-c.com/langc-headerfile/)
[5]. 『gcc コンパイルオプション備忘録』, (https://qiita.com/seriru13/items/c2f5192615162c4c3f47)

6. 演習問題解答例

6.1.　2 章演習問題

【1】

以下の条件を満たすプログラム「count_sentence.c」を作成するために、網掛け部分を埋めてみましょう。

・文字列をキーボードから入力して、配列に格納する

・その文字列を画面に表示する

・その文字列の文字数をカウントし、表示する

＜count_sentence.c＞

```
#include <stdio.h>

int main(void)
{
        char str[100];
        int count;
        //入力した文字列を配列に格納
        printf( “文字を入力してください：”);
        scanf( “%s”, &str);
        //入力した文字列を表示
        printf( “文字列は「%s」です¥n”, str);
        //str の指定した要素に文字が格納されている場合のみ実行
        count = 0 ;
        while( str[count] != ‘¥0’){
                count++;
        }
        printf( “文字数は%d です¥n”, count);
        return 0;
}
```

＜実行結果イメージ＞

```
文字を入力してください：hello
文字列は「hello」です
文字数は5です
```

【2】

以下の条件を満たすプログラム「kuku.c」を作成するために、網掛け部分を埋めてみましょう。

・2 次元配列 kuku に、九九の計算結果を格納する

・kuku の値を出力し、九九の表を画面に表示する

＜kuku.c＞

```
#include <stdio.h>

int main(void)
{
        int kuku[9][9];   //2 次元配列を定義
        int i, j;

        for( i = 0; i < 9; i++){
                for( j = 0; j < 9; j++){
                        kuku[i][j] = (i+1)*(j+1) ;   //2 次元配列に答えを格納
                        printf( “%3d”, kuku[i][j]);   //九九の計算結果を画面に表示
                }
                printf( “¥n”);
        }

        return 0;
}
```

6.2.　3 章演習問題

【1】

以下の条件を満たすプログラム「circle.c」を作成するために、網掛け部分を埋めてみましょう。

・半径の値を引数に取り、面積を返す関数 area_of_circle を作成する

・main 関数で area_of_circle を呼び出し、キーボードで入力した半径を持つ円の面積を表示する

＜circle.c＞

```
#include <stdio.h>
#include <math.h>
//関数定義
float area_of_circle(float radius) {
        return radius*radius* M_PI;
}
int main(void)
{
        float radius;   //半径を格納する変数を宣言
        //半径の値をキーボードから入力
        printf( “radius = ”);
        scanf( “%f”, &radius);
        //関数を呼び出して結果を画面に表示
        printf( “面積は  %f  です¥n”, area_of_circle(radius));
        return 0;
}
```

＜実行結果イメージ＞

```
radius = 3.4
面積は 36.316811 です
```

【2】

以下の条件を満たすプログラム「sum_of_squares.c」を作成するために、網掛け部分を埋めてみましょう。

・引数の 2 乗の値を求める関数を作成する（引数は整数とする）

・3 つの引数の和を求める関数を作成する

・上で作成した 2 つの関数を使用して、キーボードから入力した 3 つの値の 2 乗和を計算し、結果を画面に表示する（例えば入力が x, y, z のとき、x^2+y^2+z^2 を計算する）

＜sum_of_squares.c＞

```
#include <stdio.h>

int jijo( int input){   //引数の 2 乗を求める関数
        return input*input ;
}
int wa_3( int input1, int input2, int input3){   //3 つの数の和を求める関数
        return input1+input2+input3;
}
int main(void)
{
        int input[3] ;   //キーボードから入力した値を格納する変数を配列で定義
        int i;
        for( i = 0; i < 3; i++){   //キーボードから 3 つの値を入力
                printf( “input%d : ”, i+1);
                scanf( “%d”, &input[i]);
        }
        //関数を呼び出して計算結果を answer に格納
        int answer = wa_3( jijo(input[0]), jijo(input[1]), jijo(input[2]));
        //結果を画面に表示
        printf( “%d^2 + %d^2 + %d^2 = %d¥n”, input[0], input[1], input[2], answer);
        return 0;
}
```

＜実行結果イメージ＞

```
input1 : 3
input2 : 4
input3 : 5
3^2 + 4^2 + 5^2 = 50
```

【3】

以下の条件を満たすプログラム「sum_of_list.c」を作成するために、網掛け部分を埋めてみましょう。

・【2】で作成した、関数 jijo 及び関数 wa_3 の機能を併せ持つ新たな関数 jijo_wa を作成する

・jijo_wa は、整数型の配列と、その配列の要素数を引数に取る

・関数 jijo_wa を使用して、キーボードから入力した 3 つの値の 2 乗和を計算し、結果を画面に表示する

＜sum_of_squares_2.c＞

```
#include <stdio.h>

int jijo_wa( int input[ ], int n){   //配列の全要素の 2 乗和を求める関数
        int i, sum;
        for( i = 0; i < n; i++){
                sum += input[i] * input[i];
        }
        return sum;
}
int main(void)
{
        int input[3] ;   //キーボードから入力した値を格納する変数を配列で定義
        int i;
        for( i = 0; i < 3; i++){   //キーボードから 3 つの値を入力
                printf( “input%d : ”, i+1);
                scanf( “%d”, &input[i]);
        }
        //関数を呼び出して計算結果を answer に格納
        int answer = jijo_wa( input, 3);
        //結果を画面に表示
        printf( “%d^2 + %d^2 + %d^2 = %d¥n”, input[0], input[1], input[2], answer);
        return 0;
}
```

＜実行結果イメージ＞

```
input1 : 3
input2 : 4
input3 : 5
3^2 + 4^2 + 5^2 = 50
```